CO1 Activity 1: Matrix Computations & Bayesian Classification

This notebook walks through:
1. Defining a small dataset and computing its covariance matrix.
2. Finding the eigenvalues and eigenvectors of the covariance matrix.
3. Performing Singular Value Decomposition (SVD).
4. Applying Bayes' Rule, Expectation, and Variance to a classification scenario.

```python
import numpy as np
import pandas as pd

# 1. Define a small synthetic data matrix (Rows = samples, Columns = features)
# Let's assume we have 5 samples and 2 features
X = np.array([
    [2.5, 2.4],
    [0.5, 0.7],
    [2.2, 2.9],
    [1.9, 2.2],
    [3.1, 3.0]
])

print("Data Matrix X:")
print(X)

# Center the data (subtract the mean of each column)
X_centered = X - np.mean(X, axis=0)
print("\nCentered Data Matrix:")
print(X_centered)
```

```
Data Matrix X:
[[2.5 2.4]
 [0.5 0.7]
 [2.2 2.9]
 [1.9 2.2]
 [3.1 3. ]]

Centered Data Matrix:
[[ 0.46  0.16]
 [-1.54 -1.54]
 [ 0.16  0.66]
 [-0.14 -0.04]
 [ 1.06  0.76]]
```

[2]
✓ 0s

```
# 2. Compute the Covariance Matrix
# Using numpy.cov (rowvar=False because columns represent variables)
cov_matrix = np.cov(X, rowvar=False)
print("Covariance Matrix:")
print(cov_matrix)
```

```
Covariance Matrix:
[[0.938  0.8405]
 [0.8405 0.853 ]]
```

[3]
✓ 0s

```
# 3. Compute Eigenvalues and Eigenvectors
eigenvalues, eigenvectors = np.linalg.eig(cov_matrix)

print("Eigenvalues:")
print(eigenvalues)
print("\nEigenvectors (as columns):")
print(eigenvectors)
```

```
Eigenvalues:
[1.73707382 0.05392618]

Eigenvectors (as columns):
[[ 0.72474155 -0.68902082]
 [ 0.68902082  0.72474155]]
```

[4]
✓ 0s

```
# 4. Compute Singular Value Decomposition (SVD)
# X = U * S * Vt
U, S, Vt = np.linalg.svd(X_centered, full_matrices=False)

print("U matrix (Left Singular Vectors):")
print(U)
print("\nSingular Values (S):")
print(S)
print("\nVt matrix (Right Singular Vectors):")
print(Vt)
```

```
U matrix (Left Singular Vectors):
[[-0.16829698 -0.43275954]
 [ 0.82595808 -0.11844349]
 [-0.2165101   0.79253709]
 [ 0.04894784  0.14527866]
 [-0.49009883 -0.38661272]]

Singular Values (S):
[2.63596193 0.46444021]

Vt matrix (Right Singular Vectors):
[[-0.72474155 -0.68902082]
 [-0.68902082  0.72474155]]
```

```python
import scipy.stats as stats
# Class Priors
p_c1 = 0.4
p_c2 = 0.6
# Parameters for conditional likelihoods
mu_1, var_1 = 5, 1.5
mu_2, var_2 = 2, 1.0
# Let's classify a new observed data point x_obs
x_obs = 3.5

# Compute likelihoods P(x_obs | C_i)
likelihood_c1 = stats.norm.pdf(x_obs, loc=mu_1, scale=np.sqrt(var_1))
likelihood_c2 = stats.norm.pdf(x_obs, loc=mu_2, scale=np.sqrt(var_2))

# Compute marginal likelihood P(x_obs) = sum_i P(x_obs | C_i) * P(C_i)
evidence = (likelihood_c1 * p_c1) + (likelihood_c2 * p_c2)

# Compute posterior probabilities using Bayes' Rule
posterior_c1 = (likelihood_c1 * p_c1) / evidence
posterior_c2 = (likelihood_c2 * p_c2) / evidence

print(f"For observed feature value x = {x_obs}:")
print(f"Posterior Probability P(C1 | x) [Spam]: {posterior_c1:.4f}")
print(f"Posterior Probability P(C2 | x) [Ham]: {posterior_c2:.4f}")

predicted_class = "C1 (Spam)" if posterior_c1 > posterior_c2 else "C2 (Ham)"
print(f"Predicted Class: {predicted_class}")
```

```
For observed feature value x = 3.5:
Posterior Probability P(C1 | x) [Spam]: 0.4420
Posterior Probability P(C2 | x) [Ham]: 0.5580
Predicted Class: C2 (Ham)
```

- **Total Expectation**: $E[X] = E[E[X|C]] = P(C_1)\mu_1 + P(C_2)\mu_2$
- **Total Variance**: $Var(X) = E[Var(X|C)] + Var(E[X|C])$

```python
# Expectation
overall_expectation = (p_c1 * mu_1) + (p_c2 * mu_2)

# Variance
# E[Var(X|C)] (Within-class variance)
within_class_var = (p_c1 * var_1) + (p_c2 * var_2)

# Var(E[X|C]) (Between-class variance)
between_class_var = (p_c1 * (mu_1 - overall_expectation)**2) + (p_c2 * (mu_2 - overall_expectation)**2)

overall_variance = within_class_var + between_class_var

print(f"Overall Expected Value E[X]: {overall_expectation:.4f}")
print(f"Within-class Variance: {within_class_var:.4f}")
print(f"Between-class Variance: {between_class_var:.4f}")
print(f"Overall Population Variance Var(X): {overall_variance:.4f}")
```

```
Overall Expected Value E[X]: 3.2000
Within-class Variance: 1.2000
Between-class Variance: 2.1600
Overall Population Variance Var(X): 3.3600
```